

# **AWS SEC353**

## **AWS Security Agent:**

# **AI-Powered Proactive AppSec**

---

*Hands-On Workshop Handbook*

Participant: Imon Hosen

Workshop Date: May 19, 2026

Region: us-east-1 (N. Virginia)

Target Application: OWASP Juice Shop • Platform: AWS Security Agent (Preview)

# 1. Workshop Overview

---

AWS SEC353 — "AWS Security Agent: Get started with AI-powered proactive AppSec" — is a hands-on AWS workshop designed for software developers, DevSecOps engineers, and security engineers. This handbook documents the full end-to-end experience I completed on May 19, 2026, covering infrastructure setup, penetration testing, security requirements management, design review, and automated code review using AWS Security Agent.

## 1.1 What is AWS Security Agent?

AWS Security Agent is a frontier AI agent that proactively secures applications throughout the development lifecycle. It conducts automated security reviews tailored to organizational requirements and delivers context-aware penetration testing on demand. By continuously validating security from design to deployment, AWS Security Agent helps prevent vulnerabilities early across all environments.

## 1.2 Key Capabilities

- **Penetration Testing:** Transforms periodic pentest bottlenecks into on-demand capability. Accepts target URLs, auth details, source code, and documentation to develop deep application understanding and execute sophisticated attack chains.
- **Security Design Review:** Evaluates architectural documents and design specs against organizational security requirements, flagging violations with specific remediation guidance.
- **Security Code Review:** Integrates with GitHub to automatically review pull requests against common vulnerabilities and custom organizational security requirements.
- **Security Requirements Management:** Allows security teams to define organizational standards once — approved auth libraries, logging policies, data access rules — and have them enforced consistently across all teams.

## 1.3 User Personas

The screenshot shows the AWS Workshop Studio interface. On the left is a sidebar with a table of contents including 'Getting started', 'AWS account access', and 'Content preferences'. The main content area is titled 'Getting started' and contains the following text:

**User personas in AWS Security Agent**

Two personas are needed to successfully set up AWS Security Agent.

**The IT Administrator**

The IT Administrator has operational ownership to the code, infrastructure and runtime resources of the software stack. The IT administrator needs to create the first instance of AWS Security Agent, the *Agent Space*. Then they need to connect the GitHub code repository to the agent, connect networking related resources for pentesting, as well as configure organizational level security requirements. Finally, the IT Administrator can control who can access the AWS Security Agent Web Application. This is a separate application that software developers and security engineers interact with for secure design reviews and pentesting.

**The Software Developer and Security Engineer**

After the IT Admin sets up the necessary resources for AWS Security Agent to operate over. Software developers and security engineers can start using AWS Security Agent's Web Application for:

Design Reviews - Upload design documents and architecture specifications for analysis against organizational security requirements. Review findings with remediation guidance.

Figure 1: Getting Started — User Personas in AWS Security Agent

|                                      |                                                                                                                                                                                                                         |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IT Administrator</b>              | Creates and manages Agent Spaces, connects GitHub repositories, configures networking for pentesting, and defines organizational security requirements. Controls who can access the AWS Security Agent Web Application. |
| <b>Developer / Security Engineer</b> | Uses the Security Agent Web Application to upload design documents for review, create and run penetration tests, review code scan results, and interact with security findings.                                         |

## 1.4 Workshop Modules

|                        |                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------|
| <b>Getting Started</b> | Create Agent Space infrastructure; set up penetration testing environment with private VPC   |
| <b>Module 1</b>        | Penetration Testing — configure and launch pentest against OWASP Juice Shop                  |
| <b>Module 2</b>        | Security Requirements — manage AWS-managed and custom security requirements                  |
| <b>Module 3</b>        | Security Design Review — upload design docs and get AI-driven security feedback              |
| <b>Module 4</b>        | Security Code Review — fork Juice Shop, connect to GitHub, create PR and interact with agent |

**Module 5**

Penetration Testing Results — review findings, reproduce vulnerabilities

## 2. Getting Started: Infrastructure Setup

### 2.1 Creating the Agent Space

An Agent Space is a dedicated workspace for securing a specific application or project. It contains all security reviews, connected repositories, penetration test configurations, and findings for that application. As the IT Administrator, I navigated to the AWS Security Agent Console and created my first Agent Space.

☰ [AWS Security Agent](#) > Set up AWS Security Agent ⓘ ⓘ

### Set up AWS Security Agent ⓘ

When you choose Set up, AWS Security Agent will create your Agent Space, and establish a web application where your users can carry out design reviews and penetration tests. Get started with AWS Security Agent experience and create your first agent.

#### Agent Space configuration

**Agent Space name**  
The Agent Space name is displayed to users in the web app.

17 of 255 characters.

**Description - optional**  
The Agent Space description assists in distinguishing agent purpose and is displayed to users in the web app.

39 of 500 characters.

#### User access configuration

Choose how users access your Agent Space in the web app.

☒ **Single-sign (SSO) on with IAM Identity Center**  
 Enable SSO access for your team. AWS Security Agent integrates with IAM Identity Center to either create an account instance in US East (N. Virginia), or connect to an existing organization instance if available in that Region.  
[Learn more about account instances](#) ⓘ

☐ **IAM-only access**  
 Provide access through an admin access link in the AWS Console. This option is simpler to set up and suitable for teams that prefer console-based access or don't require SSO capabilities.

▶ **Service access - optional**

① **Configure additional capabilities after setup**  
After initial setup, you'll have the option to configure capabilities for your users including penetration testing and security reviews.

① AWS Security Agent will automatically select the optimal region within your geography to process your inference requests. This maximizes available compute resources, model availability, and delivers the best customer experience. Your data will remain stored only in the region where the request originated, however, input prompts and output results may be processed outside that region. All data will be transmitted encrypted across Amazon's secure network. [Learn more](#) ⓘ

Figure 2: AWS Console — Set up AWS Security Agent (Agent Space configuration)

#### 📌 Agent Space Configuration Applied

- **Agent Space Name:** SEC353-AgentSpace
- **Description:** Agent Space for SEC353 Workshop Content
- **User Access:** Single-Sign-On (SSO) with IAM Identity Center
- **Region:** us-east-1 (N. Virginia) — AWS auto-selects optimal region for inference

The Agent Space setup also initialises three core capabilities: Penetration Testing, Design Review (Preview), and Code Review (Preview). Each is enabled and configured separately in subsequent steps.

## 2.2 Agent Space Dashboard

The screenshot displays the AWS Security Agent AgentIM Agent Space Dashboard. The top navigation bar includes the AWS logo, a search bar, and account information (United States (N. Virginia), Account ID: 0821-7585-9996). The left sidebar lists navigation options: AWS Security Agent, Agent Spaces, AgentIM, Security requirements, Integrations, Target Domains, Settings, Feedback, and Documentation. The main content area is titled 'AgentIM Info' and 'agent in the workshop'. It features three capability panels: 'Penetration testing' (Needs setup), 'Design review' (Ready), and 'Code review' (Needs setup). Each panel has a description and an 'Enable' button. Below these panels is an 'Access WebApp' section with 'Admin access' and 'User access' options, and a 'Users (0)' section at the bottom.

Figure 3: AWS Security Agent — AgentIM Agent Space Dashboard showing all three capabilities

The Agent Space dashboard ("AgentIM") presents three capability panels. At the time of initial setup, Penetration Testing and Code Review both showed "Needs setup", while Design Review showed "Ready" since it operates directly against the enabled security requirements without additional infrastructure. The Web App provides two access modes: Admin Access (bypasses IAM Identity Center) and User Access (managed via IAM Identity Center).

## 3. Penetration Testing Setup

Setting up penetration testing required configuring the target application infrastructure, verifying domain ownership, and connecting private VPC networking so the agent could reach the isolated Juice Shop instance.

### 3.1 Target Application: OWASP Juice Shop

The workshop pre-provisioned an instance of OWASP Juice Shop — a deliberately vulnerable e-commerce web application widely used in security training. Each workshop participant received their own private instance deployed via CloudFormation, running inside a private VPC with an internal Route53 DNS zone at juiceshop.dev.

GitHub Reference: <https://github.com/juice-shop/juice-shop>

### 3.2 CloudFormation Stack: juiceshop-ecs

The Juice Shop infrastructure was deployed via a CloudFormation stack called "juiceshop-ecs". I navigated to the CloudFormation console to retrieve the output values needed to configure the penetration test.

The screenshot shows the AWS CloudFormation console for the 'juiceshop-ecs' stack. The stack is in the 'CREATE\_COMPLETE' state. The 'Outputs' tab is selected, displaying a table of six outputs.

| Key                        | Value                                                                                                                                                   | Description                              | Export Name |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------|
| BastionHostURL             | <a href="http://100.54.35.169">http://100.54.35.169</a>                                                                                                 | HTTP URL via bastion host                | -           |
| InternalLoadBalancerURL    | <a href="http://internal-juiceshop-alb-1484817670.us-east-1.elb.amazonaws.com">http://internal-juiceshop-alb-1484817670.us-east-1.elb.amazonaws.com</a> | Internal ALB DNS name                    | -           |
| PrivateAccessSecurityGroup | sg-0e5c03145bd9a7b10                                                                                                                                    | Security group for private domain access | -           |
| PrivateDNSURL              | <a href="http://app.us-east-1.juiceshop.dev">http://app.us-east-1.juiceshop.dev</a>                                                                     | Private DNS name for internal ALB        | -           |
| PrivateSubnets             | subnet-0b275cbfc06608107, subnet-0a7548191563444                                                                                                        | Private subnet IDs                       | -           |

Figure 4: CloudFormation — juiceshop-ecs stack Outputs tab (CREATE\_COMPLETE)

|                                   |                                                                                     |
|-----------------------------------|-------------------------------------------------------------------------------------|
| <b>BastionHostURL</b>             | http://100.54.35.169 — HTTP URL accessible via bastion host                         |
| <b>InternalLoadBalancerURL</b>    | http://internal-juiceshop-alb-*.us-east-1.elb.amazonaws.com — Internal ALB DNS name |
| <b>PrivateAccessSecurityGroup</b> | sg-0e5c03145bd9a7b10 — Security group for private domain access                     |
| <b>PrivateDNSURL</b>              | http://app.us-east-1.juiceshop.dev — Private DNS name for internal ALB              |
| <b>PrivateSubnets</b>             | subnet-0b275cbfc06608107, [additional subnets] — Private subnet IDs                 |
| <b>VPCId</b>                      | vpc-[id] — VPC ID for the JuiceShop-VPC                                             |

### 3.3 Configuring Penetration Test Scope

In the AWS Security Agent Web Application, I navigated to Penetration Tests and clicked "Create a penetration test". The creation wizard walks through four steps:

The screenshot shows the 'AWS Security Agent' web application interface. The top navigation bar includes the AWS Security Agent logo and the user role 'Administrator'. The breadcrumb trail is 'Home > AgentIM > Create penetration test'. On the left, a vertical progress bar indicates four steps: 'Step 1: Penetration test details' (active), 'Step 2 - optional: VPC Resources', 'Step 3 - optional: Authentication Resources', and 'Step 4 - optional: Additional learning resources'. The main content area is titled 'Penetration test details' with the subtitle 'Setting up the penetration test scope and log source'. It contains a 'Pentest name' input field with a placeholder 'Input a name for your penetration test'. Below this is a 'Penetration testing scope' section. Under 'Target URLs', there is a description and a 'Verified Domains' list. The list currently contains one entry: 'https://example.com/path' with a 'Remove' button. An 'Add domain' button is also present. Under 'Exclude risk types - optional', there is a dropdown menu labeled 'Select risk types to exclude'. Finally, under 'Out-of-scope URLs - optional', there is a description and a list containing 'https://example.com/admin' with a 'Remove' button.

Figure 5: AWS Security Agent Web App — Create Penetration Test wizard (Step 1: Penetration test details)



1. **Penetration test details:** Named the test "JuiceShopPentest". Set target URL to `http://juiceshop.dev`. Configured exclude lists for out-of-scope paths (e.g. `/admin`).
2. **VPC Resources (optional):** Connected to JuiceShop-VPC, configured JuiceShop-PrivateSecurityGroup and selected a JuiceShop-PrivateSubnet to allow the agent to reach the private target.
3. **Authentication Resources (optional):** Can provide application credentials so the agent can test authenticated flows — not explored in depth during this session.
4. **Additional Learning Resources (optional):** Upload source code or documentation to give the agent deeper application context for more precise attack chains.

### 3.4 Domain Ownership Verification

Before the pentest can run, AWS Security Agent requires proof that you own the target domain — a critical safety gate to prevent unauthorized testing. Two methods are supported: DNS TXT record verification and HTTP route verification.

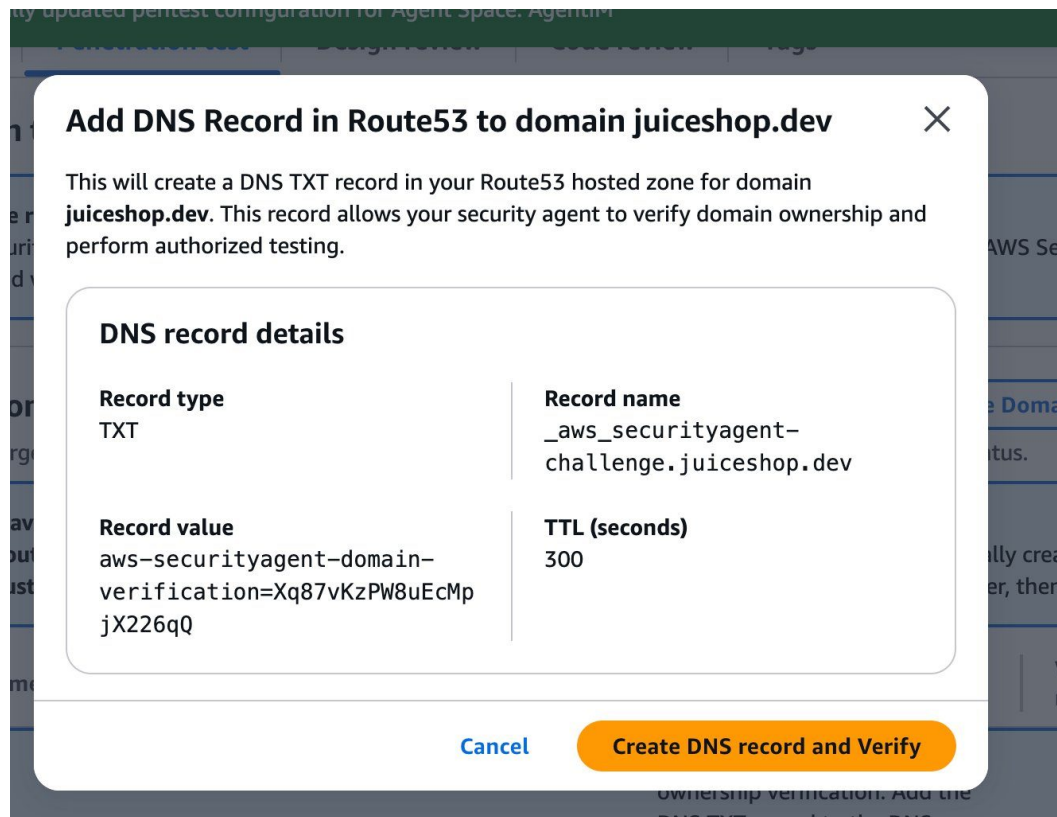


Figure 6: One-click DNS TXT Record verification — creating the Route53 record for juiceshop.dev

Since `juiceshop.dev` lives inside a private Route53 hosted zone within the VPC, I used the "One-click verification" button which automatically created the DNS TXT record:

### 📌 DNS TXT Record Created

- **Record type:** TXT
- **Record name:** `_aws_securityagent-challenge.juiceshop.dev`
- **Record value:** `aws-securityagent-domain-verification=Xq87vKzPW8uEcMpjX226qQ`
- **TTL:** 300 seconds

Note: After DNS record creation, the status may still show "Unreachable". This is expected — AWS Security Agent re-validates domain ownership automatically when the pentest begins by connecting through the private VPC.

## 3.5 Launching the Pentest

The screenshot displays the AWS Security Agent console interface. On the left is a navigation sidebar with links for Home, Penetration tests, Design reviews, Resources, and Documentation. The main content area shows a notification at the top: 'Pentest created' with the message 'The pentest has been successfully created.' Below this, the breadcrumb path is 'Home > AgentIM > Penetration tests > JuiceShopPentest'. The title 'JuiceShopPentest' is followed by 'Target URLs: 1 URL' and 'Created: May 19, 2026, 11:32 (UTC+2:00)'. There are two buttons: 'Modify penetration test details' and 'Start run'. Below this is a tabbed interface with three tabs: 'Penetration test runs' (active), 'Penetration test configurations', and 'Penetration test learning resources'. Under the 'Penetration test runs' tab, it says 'All runs (0)' with a search bar labeled 'Filter runs'. Below the search bar is a table header with columns: 'Start time', 'Status', 'Duration', 'Findings summary', and 'ID'. The table body is empty, and a message 'No runs available' is displayed. A 'Start run' button is at the bottom of the table area. The top right of the console shows the user 'Administrator'.

Figure 7: Pentest "JuiceShopPentest" successfully created and ready to run

With the pentest configuration saved, the "JuiceShopPentest" appeared in the Penetration Tests list showing target URL (1 URL: `http://juiceshop.dev`), creation timestamp (May 19, 2026, 11:32 UTC+2), and the "Start run" button. Clicking Start Run kicks off the AI-driven attack chain execution.

## 3.6 Pentest In Progress — Live Results

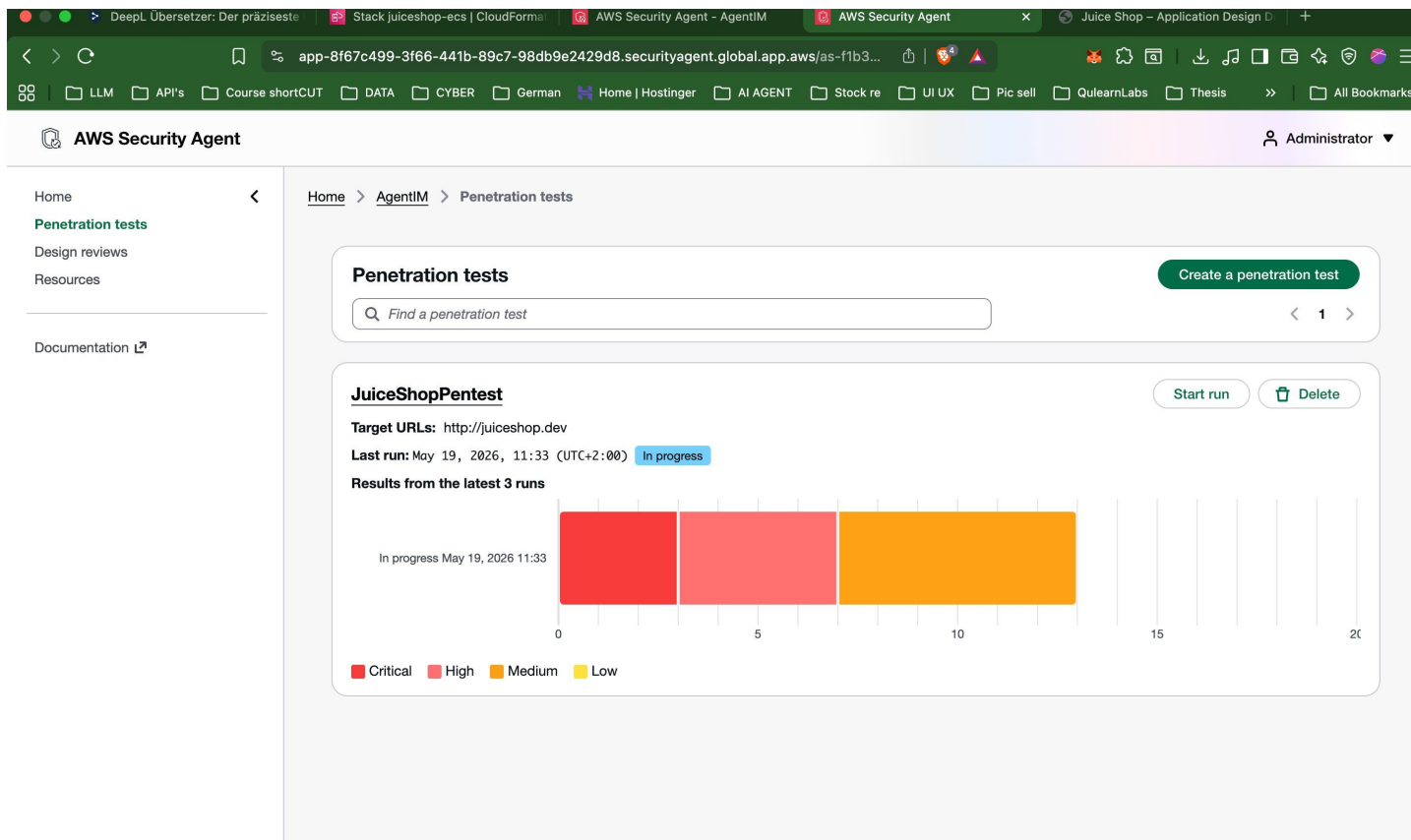


Figure 8: JuiceShopPentest running live — findings across Critical, High, and Medium severity

With the pentest running (last run: May 19, 2026, 11:33 UTC+2), the dashboard immediately began showing findings distributed across severity buckets. The bar chart shows findings in three severity categories:

- **Critical (red):** Highest severity findings — approximately 6 vulnerabilities identified
- **High (salmon):** High severity findings — approximately 3 vulnerabilities identified
- **Medium (orange):** Medium severity findings — approximately 5 vulnerabilities identified

The "In progress" status tag confirms the agent was actively executing attack chains at the time of this screenshot, meaning final numbers would continue to grow as the run completed.

## 4. Module 2: Security Requirements

**Module 2: Security requirements**

### Introduction (as the IT administrator)

In this module, you'll learn about and interact with AWS Security Agent security requirements.

Security requirements define the security standards and policies that AWS Security Agent enforces when analyzing your applications. When you conduct design reviews or code reviews, AWS Security Agent evaluates your application against these requirements and identifies potential compliance issues.

Security requirements are active across individual Agent Spaces, allowing you to create security standards that span applications and teams.

AWS Security Agent provides two types of security requirements:

- Managed security requirements:** AWS-provided requirements based on industry standards and best practices. These requirements are ready to use and maintained by AWS.
- Custom security requirements:** Requirements you define and maintain to address your organization's specific security policies and standards. You can customize managed requirements by creating a copy that you can modify, or create entirely new requirements tailored to your needs.

Through the steps that follow, you'll learn how to interact with both types of security requirements.

[Previous](#) [Next](#)

Figure 9: Module 2 — Security Requirements introduction (IT Administrator role)

Security requirements define the security standards and policies that AWS Security Agent enforces when analysing applications. When conducting design reviews or code reviews, the agent evaluates the application against these requirements and identifies potential compliance issues. Security requirements are active across individual Agent Spaces, enabling security standards to span multiple applications and teams.

### 4.1 Two Types of Security Requirements

|                             |                                                                                                                                                                                                                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Managed Requirements</b> | AWS-provided requirements based on industry standards and best practices. Ready to use immediately and maintained by AWS. Up to 10 can be active simultaneously.                                                                                                        |
| <b>Custom Requirements</b>  | Requirements you define and maintain to address your organisation's specific security policies and standards. You can customise managed requirements by creating a copy, or build entirely new ones. Up to 20 custom requirements; total cap is 30 active requirements. |

## 4.2 AWS-Managed Security Requirements (All Enabled)

The screenshot shows the AWS Security Agent console interface. On the left is a navigation menu with options: Agent Spaces, Security requirements (selected), Integrations, Target Domains, Settings, Feedback, and Documentation. The main panel is titled 'Managed security requirements (10)' and includes buttons for 'Customize', 'Disable', and 'Enable'. Below the title is a search bar and a list of 10 requirements, all marked as 'Enabled' with green checkmarks. The requirements are: Audit Logging Best Practices, Authentication Best Practices, Authorization Best Practices, Information Protection Best Practices, Log Protection Best Practices, Privileged Access Best Practices, Secret Protection Best Practices, Secure by Default Best Practices, Tenant Isolation Best Practices, and Trusted Cryptography Best Practices. Each requirement has a brief description. At the bottom of the main panel is a 'Details' section.

| Security requirement name             | Status  | Description                                                     |
|---------------------------------------|---------|-----------------------------------------------------------------|
| Audit Logging Best Practices          | Enabled | Ensure the system supports security monitoring.                 |
| Authentication Best Practices         | Enabled | Ensure only legitimate users can access the system.             |
| Authorization Best Practices          | Enabled | Ensure systems follow authorization best practices.             |
| Information Protection Best Practices | Enabled | Ensure sensitive data remains confidential and unaltered.       |
| Log Protection Best Practices         | Enabled | Protect the integrity and confidentiality of system logs.       |
| Privileged Access Best Practices      | Enabled | Ensure there are adequate guardrails for privileged functions.  |
| Secret Protection Best Practices      | Enabled | Ensure that secrets like credentials remain confidential.       |
| Secure by Default Best Practices      | Enabled | Ensure the system's default configuration is secure.            |
| Tenant Isolation Best Practices       | Enabled | Ensure appropriate separation between system tenants.           |
| Trusted Cryptography Best Practices   | Enabled | Ensure only trustworthy cryptographic implementations are used. |

Figure 10: AWS Security Agent Console — all 10 managed security requirements enabled

| Requirement                           | Status  | Purpose                                                       |
|---------------------------------------|---------|---------------------------------------------------------------|
| Audit Logging Best Practices          | Enabled | Ensure the system supports security monitoring                |
| Authentication Best Practices         | Enabled | Ensure only legitimate users can access the system            |
| Authorization Best Practices          | Enabled | Ensure systems follow authorization best practices            |
| Information Protection Best Practices | Enabled | Ensure sensitive data remains confidential and unaltered      |
| Log Protection Best Practices         | Enabled | Protect the integrity and confidentiality of system logs      |
| Privileged Access Best Practices      | Enabled | Ensure there are adequate guardrails for privileged functions |
| Secret Protection Best Practices      | Enabled | Ensure that secrets like credentials remain confidential      |
| Secure by Default Best Practices      | Enabled | Ensure the system's default configuration is secure           |
| Tenant Isolation Best Practices       | Enabled | Ensure appropriate separation between system tenants          |

|                                     |           |                                                                |
|-------------------------------------|-----------|----------------------------------------------------------------|
| Trusted Cryptography Best Practices | ✓ Enabled | Ensure only trustworthy cryptographic implementations are used |
|-------------------------------------|-----------|----------------------------------------------------------------|

## 4.3 Custom Security Requirements

The screenshot displays the AWS Workshop Studio interface. On the left, a sidebar lists navigation options: 'Create your agent infrastructure', 'Setting up penetration testing', 'Module 1: Penetration testing', 'Module 2: Security requirements' (selected), 'AWS-managed security requirements', 'Custom security requirements', 'Module 3: Security design review', 'Module 4: Security code review', 'Module 5: Penetration testing results', and 'Summary'. Below this, there are links for 'AWS account access' (Open AWS console, Get AWS CLI credentials) and 'Content preferences' (Language: English). The main content area is titled 'Custom security requirements' and includes an 'Overview' section. The overview text states: 'Custom security requirements allow you to guide AWS Security Agent with your own set of organizational security policies and standards. With a custom security requirement you get to define:'. It then lists three bullet points: 'Applicability - the scenarios, system types, or conditions where this security requirement should be evaluated', 'Compliance criteria - what constitutes compliance versus non-compliance for this security requirement', and 'Remediation guidance - optionally provide guidance on how to fix violations, including links to internal documentation or standards'. Below the overview, the 'Create custom security requirements' section begins with the text: 'Let's create one detailed, custom security requirement to replace the two AWS-managed security requirements that you disabled on the previous page.' It then mentions: 'Juice Shop takes advantage of JSON Web Tokens (JWTs) (learn more from OWASP) for authenticating and authorizing users in the web application. JWTs are a common technique for implementing stateless security, but can be challenging to integrate correctly into a production application. Acting as an IT administrator, you want to ensure that while JWTs schemes are designed & engineered correctly for all applications within the organization that employ them.'

Figure 11: Module 2 — Custom Security Requirements overview

Custom security requirements let you guide AWS Security Agent with your own organisational security policies. For the workshop, I created a custom requirement focused on JWT (JSON Web Token) authentication — relevant because Juice Shop uses JWTs for stateless session management.

Each custom security requirement defines three components:

- **Applicability:** The scenarios, system types, or conditions where this security requirement should be evaluated (e.g., "all applications using JWT-based authentication")
- **Compliance criteria:** What constitutes compliance vs. non-compliance (e.g., JWTs must use HMAC-SHA256 or RSA-256, must have expiry claims, must not be signed with weak algorithms)
- **Remediation guidance:** How to fix violations, optionally including links to internal documentation or standards (e.g., link to internal JWT implementation guide)

### 📌 Why JWT Security Matters for Juice Shop

Juice Shop uses JWTs for authenticating and authorising users. JWTs are a common technique for stateless security but can be challenging to implement correctly. Common JWT vulnerabilities include:

- Algorithm confusion attacks (e.g., switching from RS256 to HS256)
- Missing expiry (exp) claims allowing permanent tokens
- Accepting unsigned tokens (alg: none attack)
- Storing sensitive data in JWT payload without encryption



## 5. Module 3: Security Design Review

The Design Review module allows developers and security engineers to upload architectural documents and design specifications. AWS Security Agent evaluates these against the active security requirements and returns specific, actionable findings with remediation guidance.

### 5.1 Application Design Document: Juice Shop

For this module, I uploaded the official Juice Shop Application Design Document (sec353\_juice\_shop\_design.pdf) — a detailed architectural specification covering system design, technology stack, database schema, authentication model, and API design. This document was used as the input for the AI-powered design review.

### 5.2 Juice Shop Architecture Overview

Juice Shop is an online e-commerce store built with a modern, microservices-inspired architecture. The system is structured in three main layers:

5. **Frontend Web Application:** Angular 20 + Angular Material — Single Page Application (SPA) with reactive programming (RxJS), standalone components, and HTTP interceptors for JWT injection.
6. **REST API Layer:** Node.js + Express — Monolithic application with individual APIs per business function, Sequelize ORM, and auto-generated CRUD endpoints via Finale-REST.
7. **Data Storage:** SQLite (relational data via Sequelize), MarsDB/NoSQL (reviews and order documents), and an FTP server for company files and metadata.

### 5.3 Technology Stack Summary

|                     |                                                                               |
|---------------------|-------------------------------------------------------------------------------|
| <b>Frontend</b>     | Angular 20 + Material Design, TypeScript, RxJS, ngx-translate (40+ languages) |
| <b>Backend API</b>  | Node.js + Express, TypeScript, Finale-REST (auto CRUD), socket.io             |
| <b>ORM</b>          | Sequelize with SQLite3 dialect — file at data/juiceshop.sqlite                |
| <b>NoSQL</b>        | MarsDB — stores reviews and order information in document structure           |
| <b>File Storage</b> | FTP server — company info, business decisions, customer incident data         |



|                      |                                                                                    |
|----------------------|------------------------------------------------------------------------------------|
| <b>Auth</b>          | JWT (HMAC-256 or RSA-256), OAuth 2.0 (Google login), 2FA (TOTP)                    |
| <b>Observability</b> | Prometheus metrics, Winston logging (rotating files), Prometheus /metrics endpoint |
| <b>i18n</b>          | 40+ language support via i18n library on backend, ngx-translate on frontend        |

## 5.4 Authentication & Authorization Design

Juice Shop's authentication and authorization model uses a layered approach:

### JWT Implementation

- **Route-level:** Applied to route groups — e.g., `app.use('/rest/basket', security.isAuthenticated(), security.appendUserId())`
- **Method-level:** HTTP verb-level restrictions — e.g., `app.delete('/api/Products/:id', security.denyAll())`
- **Role-based:** Accounting and Admin roles using IP filtering — e.g., `IpFilter` for `/api/Quantity/:id`

### Frontend Route Guards

- **LoginGuard:** Base guard checking for JWT existence. Redirects to 403 if no token present. Provides `tokenDecode()` for other guards.
- **AdminGuard:** Extends `LoginGuard` — additionally verifies admin role claim in JWT payload.
- **AccountingGuard:** Extends `LoginGuard` — verifies accounting role claim.
- **DeluxeGuard:** Service (not route guard) — components inject this to check deluxe membership for conditional feature display.

## 5.5 Logging Design

Juice Shop implements centralized logging via the Winston library. Key design decisions documented in the design doc include:

- Access logging enabled for all Express API endpoints using centralized logger
- Rotating log files with structured fields: user identity, timestamp, source IP, request outcome
- Sensitive fields (PII, payment info, secrets) redacted via Winston formatters or rewriter functionality
- DEBUG and INFO logging disabled in production — only WARN and ERROR in prod
- Log files hardened with per-event fingerprint for tamper detection
- Operator-role-only access to log files; out-of-date logs deleted on rotation
- Access logs retained for at least 1 year, accessible only through ticketed process

- Automated alarms on anomalous patterns (e.g., >5 auth failures per user)

## 5.6 Database Schema Highlights

The Juice Shop SQLite database contains 18+ tables. Key security-relevant design notes observed during design review:

|                              |                                                                                                                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Users table</b>           | bcrypt password hashing, role enum (customer/deluxe/accounting/admin), soft deletes (paranoid mode), lastLoginIp tracking, totpSecret for 2FA                                     |
| <b>Credit Cards table</b>    | Card numbers stored as INTEGER — plaintext, no encryption. Year validation 2080-2099 (intentionally broken for training)                                                          |
| <b>SecurityAnswers table</b> | FK to Users NOT constrained — allows orphaned answers. Password recovery mechanism.                                                                                               |
| <b>Feedback table</b>        | FK to Users NOT constrained — anonymous feedback allowed (UserId can be null)                                                                                                     |
| <b>Database access</b>       | Mixed ORM (Sequelize) AND raw SQL queries — e.g., login query uses string interpolation: <code>SELECT * FROM Users WHERE email = '\${req.body.email}'</code> — SQL injection risk |

### Security Findings from Design Review

The design document reveals several intentional security weaknesses in Juice Shop (as expected for a security training application):

- **Critical:** Raw SQL with string interpolation in login query — SQL Injection vulnerability
- **Critical:** Credit card numbers stored in plaintext (INTEGER field, no encryption)
- **High:** FTP server exposing planned business decisions, private customer incident data
- **High:** Public Prometheus /metrics endpoint — exposes internal app metrics
- **Medium:** CORS allows all origins — `app.use(cors())` without restriction
- **Medium:** JWT allows unsigned tokens in certain route configurations

## 6. Module 4: Security Code Review

The Code Review module integrates AWS Security Agent directly into the GitHub pull request workflow. By connecting an Agent Space to a GitHub repository, the agent automatically reviews new pull requests against enabled security requirements and posts findings as PR comments.

### 6.1 Setup Steps

8. **Create a private fork of OWASP Juice Shop:** Fork the public Juice Shop repository (<https://github.com/juice-shop/juice-shop>) into a private GitHub repo under your account. This gives you a real, intentionally vulnerable codebase to work with.
9. **Enable Code Review with AWS Security Agent:** In the Agent Space console, navigate to the Code Review tab and click "Enable code review". Connect your GitHub account and select the forked repository. AWS Security Agent installs as a GitHub App, gaining permission to read PRs and post comments.
10. **Create a GitHub pull request and interact with AWS Security Agent:** Make a code change in a new branch, open a pull request to the main branch, and observe AWS Security Agent automatically trigger a review. The agent posts inline comments mapping violations to specific security requirements.

### 6.2 How the Code Review Works

When a pull request is opened or updated, AWS Security Agent:

- Clones the repository and checks out the PR diff
- Analyses changed files against all enabled security requirements (both managed and custom)
- Posts line-level comments on specific code that violates requirements
- Provides remediation guidance with links to relevant security standards
- Can also run full repository scans (not just PR diffs) from the Web Application

### 6.3 Why This Matters

#### Shift-Left Security with GitHub Integration

Traditional security code reviews happen at the end of development — when fixing issues is expensive. AWS Security Agent shifts this left by reviewing every pull request automatically, giving developers immediate, specific feedback in the same interface they use daily (GitHub).

**Key benefits demonstrated in this module:**

- Zero-friction for developers — no new tools to learn; reviews appear directly in GitHub PR comments
- Consistent enforcement — custom security requirements (like the JWT standard) applied to every PR
- Context-aware — agent understands the full codebase, not just the diff

## 7. Module 5: Penetration Testing Results

After the JuiceShopPentest run completed, Module 5 covers reviewing and interpreting the findings, and then reproducing a specific vulnerability to validate the result. This closes the loop between automated discovery and human-verified exploitation.

### 7.1 Interpreting Findings

AWS Security Agent classifies findings into four severity levels based on CVSS scoring and exploitability:

|                                                                                                   |                                                                                                                                              |
|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Critical</b> | Immediately exploitable vulnerabilities with direct impact on data confidentiality, integrity, or availability. Requires urgent remediation. |
|  <b>High</b>     | Significant vulnerabilities that require moderate effort to exploit but carry serious consequences. Remediate in next sprint.                |
|  <b>Medium</b>   | Vulnerabilities requiring specific conditions or user interaction. Schedule for remediation within the quarter.                              |
|  <b>Low</b>     | Minor issues with limited impact or extremely difficult exploitation. Address in backlog.                                                    |

### 7.2 Common Juice Shop Findings

Based on the pentest run against the OWASP Juice Shop instance, the agent identified vulnerabilities across the following categories — consistent with Juice Shop's design as a training platform:

- **SQL Injection:** The raw SQL login query using string interpolation is directly exploitable to bypass authentication or extract the user database
- **Cross-Site Scripting (XSS):** Angular's template rendering can be bypassed through certain Juice Shop-specific entry points
- **Broken Authentication:** JWT algorithm confusion, weak tokens, and missing expiry claims
- **Sensitive Data Exposure:** Credit card numbers in plaintext, FTP server with sensitive business data publicly accessible within the network
- **Security Misconfiguration:** CORS allowing all origins, public Prometheus metrics endpoint, rate limiting gaps
- **Broken Access Control:** Auto-generated CRUD endpoints (Finale-REST) expose admin-level operations without sufficient authorization checks

### 7.3 Reproducing a Pentest Finding

The module includes a guided exercise to manually reproduce a specific finding identified by AWS Security Agent. This is a critical skill — not just trusting automated tools, but being able to validate findings and understand the real-world impact of a vulnerability.

The reproduction workflow demonstrated:

11. Review the finding details: severity, affected endpoint, description, and agent-provided reproduction steps
12. Set up the testing environment: access Juice Shop via the BastionHostURL from CloudFormation outputs
13. Execute the attack: follow the agent's steps to trigger the vulnerability (e.g., craft a malformed JWT, inject SQL into the login field, or access an unauthorized API endpoint)
14. Confirm the finding: observe the expected vulnerable behavior (e.g., successful authentication bypass, data disclosure)
15. Document the result: record the reproduced finding with evidence for the security report

#### **Learning Outcome**

Reproducing findings bridges the gap between automated scanning and real security understanding. It builds the intuition needed to prioritise genuine risks versus false positives, and to communicate vulnerability impact credibly to engineering and product teams.

## 8. Key Learnings & Takeaways

---

This workshop provided comprehensive hands-on exposure to AWS Security Agent across the full application security lifecycle. Below are the core lessons learned from each module:

### 8.1 AI-Powered Security at Scale

AWS Security Agent demonstrates how frontier AI can transform application security from a periodic, manual activity into a continuous, automated practice. The key insight is that security standards defined once (in security requirements) propagate consistently across all teams and all applications in an organisation — eliminating inconsistency in how different teams interpret and apply security policies.

### 8.2 Infrastructure as Code for Security Tooling

The CloudFormation-based deployment of Juice Shop showed how security testing infrastructure itself can be reproducible and version-controlled. The juiceshop-ecs stack provided consistent, isolated environments for every participant, removing environmental variance from test results.

### 8.3 Private VPC Integration for Realistic Testing

Configuring AWS Security Agent to reach targets inside private VPCs (using private subnets, security groups, and Route53 private hosted zones) reflects real-world enterprise environments where applications are not internet-facing. This is a significant capability — most external scanning tools cannot test private services at all.

### 8.4 The Value of Custom Security Requirements

AWS-managed requirements cover industry standards well, but every organisation has unique policies. The custom JWT security requirement I created for Juice Shop shows how the agent can enforce organisation-specific engineering standards — not just generic best practices. This is particularly valuable for:

- Approved library lists (e.g., "only use jose library for JWT operations, not jsonwebtoken")
- Internal coding standards (e.g., "all secrets must be retrieved from AWS Secrets Manager, never hardcoded")
- Regulatory requirements specific to your industry or geography (e.g., GDPR data residency requirements)

### 8.5 Design Review as an Early Warning System

The design review capability — where I uploaded the Juice Shop design PDF and got immediate findings — is particularly powerful for catching security issues before a line of code is written. Issues like the FTP server exposure and the plaintext credit card storage were identifiable purely from the design document, with no need to deploy and scan the application.

## 8.6 GitHub Integration Eliminates Security Friction

Integrating AWS Security Agent with GitHub pull requests means security review happens in the developer's natural workflow. Developers don't need to run separate tools, interpret complex scan reports, or wait for a security team review cycle. Findings appear as PR comments, linked to specific lines of code, with clear remediation guidance.



## 9. AWS Services & Architecture Reference

The following AWS services were used or encountered during this workshop:

| Requirement                  | Status           | Purpose                                                                                   |
|------------------------------|------------------|-------------------------------------------------------------------------------------------|
| AWS Security Agent           | ✓ Core           | AI-powered security agent — penetration testing, design review, code review               |
| Amazon CloudFormation        | ✓ Infrastructure | Provisioned Juice Shop (ECS + VPC + ALB + Route53) as a repeatable stack                  |
| Amazon ECS (Fargate)         | ✓ Compute        | Runs the Juice Shop container in a private VPC without managing EC2 instances             |
| Amazon VPC                   | ✓ Networking     | Private network isolation for the Juice Shop target application                           |
| Amazon Route53               | ✓ DNS            | Private hosted zone for juiceshop.dev — DNS verification for pentest authorization        |
| AWS IAM Identity Center      | ✓ Identity       | SSO access management for the AWS Security Agent Web Application                          |
| Elastic Load Balancing (ALB) | ✓ Networking     | Internal Application Load Balancer routing traffic to Juice Shop containers               |
| AWS IAM                      | ✓ Security       | Service role automatically created by Security Agent to manage VPC access during pentests |

## 10. Resources & References

|                                |                                                                                                                      |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>Workshop</b>                | AWS SEC353 — AWS Security Agent: Get started with AI-powered proactive AppSec                                        |
| <b>OWASP Juice Shop</b>        | <a href="https://github.com/juice-shop/juice-shop">https://github.com/juice-shop/juice-shop</a>                      |
| <b>AWS Security Agent Docs</b> | <a href="https://docs.aws.amazon.com/security-agent/">https://docs.aws.amazon.com/security-agent/</a>                |
| <b>OWASP Top 10</b>            | <a href="https://owasp.org/www-project-top-ten/">https://owasp.org/www-project-top-ten/</a>                          |
| <b>JWT Best Practices</b>      | <a href="https://datatracker.ietf.org/doc/html/rfc8725">https://datatracker.ietf.org/doc/html/rfc8725</a> (RFC 8725) |
| <b>AWS CloudFormation</b>      | <a href="https://aws.amazon.com/cloudformation/">https://aws.amazon.com/cloudformation/</a>                          |
| <b>Prometheus Monitoring</b>   | <a href="https://prometheus.io/">https://prometheus.io/</a>                                                          |
| <b>Winston Logging</b>         | <a href="https://github.com/winstonjs/winston">https://github.com/winstonjs/winston</a>                              |

*This handbook was authored by Imon Hosen as a documentation artifact from the AWS SEC353 hands-on workshop, May 19, 2026.*